

REST Assured – Senior SDET Interview Guide

Questions + Interview-Ready Answers + Java Code + POJO + Jackson + Framework Design + CI/CD

Target: Senior SDET / QA Automation Lead / Test Automation Architect

How to prepare: Understand the answer, practise the code, and be ready to explain the design decisions behind your framework.

1. REST API Fundamentals

Q1. What is an API?

Answer: An API is a contract through which software components communicate. A REST API commonly exposes resources through HTTP endpoints and exchanges representations such as JSON.

Q2. What is REST?

Answer: REST (Representational State Transfer) is an architectural style for distributed systems. Typical REST APIs use resources, HTTP methods, stateless communication and representations such as JSON.

Q3. What are the REST constraints?

Answer: Client-server, statelessness, cacheability, uniform interface, layered system and optional code-on-demand. For SDET interviews, statelessness and uniform interface are especially important.

Q4. What is a resource?

Answer: A business entity exposed by an API, such as a user, order, policy or claim. It is commonly identified by a URI such as /users/101.

Q5. What does stateless mean?

Answer: Each request contains the information needed to process it. The server does not depend on application session state stored from previous requests.

Q6. REST vs SOAP?

Answer: REST is an architectural style commonly used with HTTP and JSON. SOAP is a protocol with a formal XML envelope and related standards. SOAP can provide strict contracts and enterprise WS-* features; REST is often simpler for web APIs.

2. HTTP Methods & Status Codes

Q7. Explain GET, POST, PUT, PATCH and DELETE.

Answer: GET retrieves data; POST commonly creates a resource or triggers an operation; PUT generally replaces a representation; PATCH partially updates; DELETE removes a resource.

Q8. What is idempotency?

Answer: An operation is idempotent when repeating the same request has the same intended server-side effect as performing it once. GET, PUT and DELETE are generally idempotent by HTTP semantics; POST is generally not.

Q9. PUT vs PATCH?

Answer: PUT is normally used for complete replacement, while PATCH applies a partial modification. The exact contract is API-specific.

Q10. 200 vs 201 vs 204?

Answer: 200 means successful processing; 201 means a resource was created; 204 means successful processing with no response content.

Q11. 401 vs 403?

Answer: 401 means authentication is missing or not accepted; 403 means the server understood the request but refuses authorization. Exact API/gateway behaviour can vary.

Q12. 400 vs 422?

Answer: 400 generally indicates a malformed or invalid request. 422 is commonly used when syntax is valid but semantic validation fails.

Q13. What is 409?

Answer: 409 Conflict indicates a conflict with the current resource state, such as a uniqueness or state constraint.

Q14. What is 429?

Answer: 429 Too Many Requests indicates rate limiting. Tests should verify documented throttling and retry behaviour.

3. REST Assured Core

Q15. What is REST Assured?

Answer: A Java DSL for testing REST APIs. It supports request construction, authentication, response validation, extraction, JSON/XML handling and integration with frameworks such as TestNG.

Q16. Explain given(), when(), then().

Answer: given() sets up the request; when() executes the operation; then() validates the response.

Q17. Basic GET example.

Answer: A minimal test is:

```
given()
    .when()
    .get("/users")
    .then()
    .statusCode(200);
```

Q18. How do you send POST?

Answer: Set content type and body, execute POST and validate the response.

```
given()
    .contentType(ContentType.JSON)
    .body(requestBody)
    .when()
    .post("/users")
    .then()
    .statusCode(201);
```

Q19. How do you pass a path parameter?

Answer: Use pathParam() with a placeholder.

```
given()
    .pathParam("id", 101)
    .when()
    .get("/users/{id}")
    .then()
    .statusCode(200);
```

Q20. How do you pass query parameters?

Answer: Use queryParams().

```
given()
    .queryParams("page", 2)
    .queryParams("limit", 10)
    .when()
    .get("/users")
    .then()
    .statusCode(200);
```

Q21. How do you add headers?

Answer: Use header() for individual headers or headers() for multiple headers.

```
given()
    .header("Authorization", "Bearer " + token)
    .header("Content-Type", "application/json")
    .when()
    .get("/users")
    .then()
    .statusCode(200);
```

4. POJO, Serialization & Deserialization

Q22. What is a POJO?

Answer: A Plain Old Java Object. In API automation it represents request or response data as a strongly typed Java model.

Q23. Why use POJO in REST Assured?

Answer: POJOs improve type safety, readability, reuse and maintainability, especially for complex request/response models.

Q24. What is serialization?

Answer: Converting a Java object such as a POJO into JSON/XML so it can be sent to an API.

Q25. What is deserialization?

Answer: Converting JSON/XML response data into a Java object such as a POJO.

Q26. How does REST Assured serialize a POJO?

Answer: When a POJO is passed to `body()`, REST Assured uses a configured object mapper such as Jackson to convert the object to the request representation.

Q27. POJO request example.

Answer: Example using an Employee model:

```
Employee employee = new Employee(
    "Swapnil",
    "SDET",
    1500000,
    Arrays.asList("Java", "Selenium", "Playwright")
);

given()
    .contentType(ContentType.JSON)
    .body(employee)
.when()
    .post("/employees")
.then()
    .statusCode(201);
```

Q28. How do you deserialize a response into a POJO?

Answer: Extract the response and use `as(Class)`.

```
Employee employee =
    given()
        .when()
            .get("/employees/101")
        .then()
            .statusCode(200)
            .extract()
            .as(Employee.class);

System.out.println(employee.getName());
```

Q29. How do you handle nested JSON with POJO?

Answer: Create a separate model for the nested object. For example, Employee contains Address address; then `employee.getAddress().getCity()` accesses the nested value.

Q30. How do you deserialize a list of POJOs?

Answer: Use REST Assured `getList` with the target class or Jackson `TypeReference`.

Q31. Why is TypeReference needed?

Answer: Java type erasure removes generic type information at runtime. `TypeReference` preserves information such as List for Jackson.

Q32. POJO vs JsonPath?

Answer: `JsonPath` is convenient for individual extraction/validation. POJO is preferable when a response is a reusable domain model or has complex nested structures.

5. Jackson & ObjectMapper

Q33. What is Jackson?

Answer: A widely used Java library for JSON serialization and deserialization.

Q34. What is ObjectMapper?

Answer: Jackson's main API for converting Java objects to JSON and JSON to Java objects.

Q35. How do you convert POJO to JSON?

Answer: `ObjectMapper mapper = new ObjectMapper();`
`String json = mapper.writeValueAsString(employee);`

Q36. How do you convert JSON to POJO?

Answer: `Employee employee =`
`mapper.readValue(json, Employee.class);`

Q37. What is @JsonProperty?

Answer: It maps a Java field/property to a JSON property name when the names differ.

```
@JsonProperty("employee_name")
private String employeeName;
```

Q38. What are @JsonIgnore, @JsonInclude and @JsonFormat?

Answer: @JsonIgnore excludes a property; @JsonInclude controls which values are included; @JsonFormat controls formatting for values such as dates.

6. Authentication & Security

Q39. What authentication types should an SDET know?

Answer: Basic Auth, API keys, Bearer tokens, OAuth 2.0, JWT, client credentials and refresh-token flows.

Q40. How do you use Basic Auth?

Answer: `given()`
`.auth().basic("username", "password")`
`.when()`
`.get("/users")`
`.then()`
`.statusCode(200);`

Q41. How do you send a Bearer token?

Answer: `given()`
`.header("Authorization", "Bearer " + token)`
`.when()`
`.get("/users")`
`.then()`
`.statusCode(200);`

Q42. How do you use OAuth2?

Answer: `given()`
`.auth().oauth2(token)`
`.when()`
`.get("/users")`
`.then()`
`.statusCode(200);`

Q43. How would you manage tokens in a framework?

Answer: Create an AuthClient/TokenManager that obtains and caches tokens according to expiry rules. Keep credentials in secure CI/CD configuration, not source code.

Q44. What negative authentication tests would you automate?

Answer: Missing token, invalid token, expired token, insufficient role/scope, malformed credentials and access to resources that the authenticated user should not be allowed to access.

7. Response Validation & JSONPath

Q45. How do you extract response data?

Answer: Response response =
given()
.when()
.get("/users/101");

String name = response.jsonPath().getString("name");

Q46. How do you extract nested data?

Answer: String city =
response.jsonPath().getString("employee.details.address.city");

Q47. How do you validate response headers?

Answer: then()
.header("Content-Type", containsString("application/json"));

Q48. How do you validate response time?

Answer: then()
.time(lessThan(2000L));

Q49. How do you validate JSON schema?

Answer: Use REST Assured's JSON schema validator with a schema file to validate response structure and types against the contract.

8. RequestSpecification & ResponseSpecification

Q50. What is RequestSpecification?

Answer: A reusable definition of common request configuration such as base URI, headers, authentication and content type.

Q51. Why use RequestSpecification?

Answer: It centralizes common setup and reduces duplication across hundreds of tests.

Q52. Give an example.

Answer: RequestSpecification requestSpec =
new RequestSpecBuilder()
.setBaseUri(BASE_URL)
.setContentType(ContentType.JSON)
.build();

given()
.spec(requestSpec)
.when()
.get("/users")
.then()
.statusCode(200);

Q53. What is ResponseSpecification?

Answer: A reusable definition of common response expectations such as status code, content type and headers.

Q54. Why use it?

Answer: It provides consistent validation and avoids repeating common assertions.

9. API Chaining & Dynamic Data

Q55. What is API chaining?

Answer: Using data returned by one API call, such as an ID or token, in a subsequent API call.

Q56. Create → Get example.

```
Answer: int userId =  
given()  
.body(requestBody)  
.when()  
.post("/users")  
.then()  
.statusCode(201)  
.extract()  
.path("id");  
  
given()  
.pathParam("id", userId)  
.when()  
.get("/users/{id}")  
.then()  
.statusCode(200);
```

Q57. How do you avoid hard-coded test data?

Answer: Generate data through APIs, use factories/builders or controlled fixtures, isolate records for parallel runs and clean up created data.

10. Negative & Edge-Case Testing

Q58. What negative scenarios should be covered?

Answer: Invalid payloads, missing mandatory fields, invalid types, invalid IDs, authentication/authorization failures, duplicates, malformed JSON, unsupported content type, rate limits and boundary values.

Q59. How do you test 404?

Answer: Request a known-invalid/non-existing resource identifier and verify the documented status and error contract.

Q60. How do you test 409?

Answer: Trigger a documented conflict such as creating a duplicate unique resource and verify status plus error structure.

Q61. What is boundary testing for APIs?

Answer: Test minimum, maximum, just-inside, just-outside, zero, negative, empty and null values wherever the contract defines constraints.

11. Multipart, Files & Headers

Q62. What is multipart/form-data?

Answer: A media type used to send multiple request parts, commonly files plus form fields.

Q63. How would you test file upload?

Answer: Use REST Assured multipart() with the file and validate the response according to the API contract.

Q64. Content-Type vs Accept?

Answer: Content-Type describes the request body format. Accept describes the response representation the client prefers.

Q65. How do you handle cookies?

Answer: Use cookie() for request cookies and extract cookies from responses when session-based APIs require them.

12. TestNG + REST Assured

Q66. Why combine REST Assured with TestNG?

Answer: TestNG provides lifecycle, groups, data providers, assertions, parallel execution and suite management while REST Assured handles HTTP operations.

Q67. How do you parameterize API tests?

Answer: Use TestNG DataProvider, external data, factories/builders or configuration-driven data.

Q68. How do you run tests in parallel safely?

Answer: Avoid shared mutable state, generate unique test data, isolate records, use thread-safe utilities and avoid static mutable response/token state.

Q69. How do you separate smoke and regression?

Answer: Use TestNG groups, suite files or Maven profiles so CI can execute a fast smoke set and broader regression independently.

13. Senior Framework Architecture

Q70. How would you design a REST Assured framework?

Answer: Use configuration, reusable request/response specifications, API clients, POJO models, authentication/token management, test-data utilities, tests, reporting and CI/CD. Keep business-level tests separate from low-level HTTP construction.

Q71. Why create API Client classes?

Answer: They encapsulate endpoints and HTTP details. Tests then express business intent such as `userClient.createUser()` rather than repeating request construction.

Q72. What belongs in BaseTest?

Answer: Only genuinely shared setup such as configuration, common specifications, hooks and reporting. Avoid business logic and test-specific state.

Q73. How do you manage environments?

Answer: Keep base URLs and environment-specific settings in external configuration and select them through CI variables/profiles. Keep secrets outside Git.

Q74. How do you manage secrets?

Answer: Use Jenkins credentials, a cloud secret manager or an approved vault. Never commit passwords, private keys or long-lived tokens.

Q75. How do you implement logging safely?

Answer: Log enough request/response information for diagnosis while masking authorization headers, passwords, tokens and sensitive personal data.

Q76. How do you make the framework maintainable?

Answer: Use single responsibility, reusable clients/specifications, typed models, central configuration, deterministic data, meaningful tests and clear reporting.

14. CI/CD & Jenkins

Q77. How do you integrate REST Assured with Jenkins?

Answer: Checkout code, configure Java/Maven, inject environment and secrets, execute the required Maven/TestNG suite, publish results/reports and archive useful logs.

Q78. Common Maven command?

Answer: `mvn clean test`

Q79. How do you run smoke vs regression in CI?

Answer: Use Maven profiles, TestNG groups or suite XML files. Fast smoke tests can run on pull requests; broader suites can run on scheduled/release pipelines.

Q80. A test passes locally but fails in Jenkins. What do you check?

Answer: Compare Java/dependency versions, environment variables, URL/network/proxy, certificates, credentials, test data, timezone, parallelism, filesystem assumptions and logs.

15. Senior Scenario Questions

Q81. An API is flaky. What do you do?

Answer: Determine whether the cause is service, environment, dependency, data or test. Analyse response codes, timings and logs. Fix the root cause where possible. Use bounded retry only for documented transient conditions.

Q82. API suite takes 40 minutes. How do you reduce time?

Answer: Parallelize independent tests, remove redundant setup, optimise data creation, use API-level setup instead of UI setup, reduce duplicate coverage and split suites by purpose.

Q83. API vs UI automation?

Answer: Validate business rules, contracts and error handling at API level where possible. Keep UI automation for critical user journeys and UI-specific behaviour.

Q84. How would you test an insurance-policy creation API?

Answer: Cover valid creation, required fields, boundaries, invalid combinations, duplicate rules, authentication/authorization, state transitions, downstream effects, response contract, persistence and cleanup.

Q85. How do you test rate limiting?

Answer: Generate controlled traffic, verify 429 behaviour and documented headers, verify recovery after the window and ensure retries do not create a retry storm.

Q86. How do you design for parallel execution?

Answer: Use unique data, avoid shared mutable objects, isolate state, use thread-safe utilities and make cleanup independent of test order.

16. Practical Coding Questions

Q87. Create a user and validate generated ID.

Answer: POST the request, extract id, assert it is present, and optionally use it in a GET request.

Q88. Validate a nested field.

Answer: Use `response.jsonPath().getString("address.city")` for a simple assertion or a POJO when the model is reused.

Q89. Validate an employee list.

Answer: Extract the list or use a JSON-path size matcher and verify the documented minimum/expected size.

Q90. Build a reusable request specification.

Answer: Use `RequestSpecBuilder` for base URI, content type, common headers and authentication, then reuse it with `.spec()`.

Q91. Implement create → update → get → delete.

Answer: Extract the created ID, pass it to each subsequent request, validate every response and clean up the test data.

17. Top 25 Questions to Revise Before the Interview

1. *What is REST and what are its constraints?*
2. *GET vs POST vs PUT vs PATCH vs DELETE?*
3. *What is idempotency?*
4. *401 vs 403?*
5. *200 vs 201 vs 204?*
6. *Explain given(), when(), then().*
7. *Path parameter vs query parameter?*
8. *What is RequestSpecification?*
9. *What is ResponseSpecification?*
10. *What is POJO and why use it?*
11. *What is serialization?*
12. *What is deserialization?*
13. *How does Jackson ObjectMapper work?*
14. *How do you serialize a POJO?*
15. *How do you deserialize a response into a POJO?*
16. *How do you handle nested POJO objects?*
17. *How do you deserialize List?*
18. *JsonPath vs POJO?*
19. *How do you handle OAuth2/Bearer tokens?*
20. *How do you implement API chaining?*
21. *How do you validate JSON schema?*
22. *How do you handle negative API scenarios?*
23. *How would you design a REST Assured framework?*
24. *How do you execute tests in parallel safely?*
25. *How do you integrate REST Assured with Jenkins?*

18. Senior SDET Answer Pattern

Use this structure for scenario questions: Requirement → Design → Implementation → Validation → Maintainability → CI/CD → Risks/Trade-offs.

Key interview point: Do not memorise only REST Assured syntax. Be ready to explain why you use POJOs, why API clients are separated from tests, how you manage tokens and dynamic data, how you make tests thread-safe, and how the framework behaves when hundreds of tests run in Jenkins.